

Almacenamiento en QEMU/KVM + libvirt

Pools, volúmenes, gestión y redimensión

✉ José Domingo Muñoz

🏫 IES Gonzalo Nazareno · Dos Hermanas

📖 IV · Infraestructura Virtual

01

Conceptos y tipos de almacenamiento

Pools, volúmenes y formatos de imagen

Conceptos básicos

POOL DE ALMACENAMIENTO

Recurso de almacenamiento gestionado por libvirt.

Tipos principales:

- `dir` — directorio del sistema de archivos
- `logical` — grupo de volúmenes LVM
- `disk` — disco físico completo
- `netfs` — directorio de un servidor NFS

VOLUMEN

Medio de almacenamiento que representa el **disco de una MV**.

Según el tipo de pool puede ser:

- Un **fichero de imagen** (`dir` , `netfs`)
- Un **volumen lógico LVM** (`logical`)
- Una **partición** del disco (`disk`)

Tipo de pool: `dir`

- El pool controla un **directorio del host** (sistema de archivos)
- Los volúmenes son **ficheros de imagen de disco**:

FORMATO	CARACTERÍSTICAS
raw	Imagen binaria de disco. Ocupa todo el espacio asignado. Acceso más eficiente. Sin snapshots ni aprovisionamiento dinámico
qcow2	Formato QEMU copy-on-write. Solo ocupa el espacio de los datos (aprovisionamiento ligero). Soporta snapshots. Algo menos eficiente
vdi, vmdk	Formatos de otros sistemas de virtualización (VirtualBox, VMware)

 El tipo `dir` **no ofrece almacenamiento compartido** entre hosts. En este curso trabajaremos principalmente con pools `dir`.

Otros tipos de pool: `logical`, `disk`, `netfs`

LOGICAL

- Controla un **grupo de volúmenes** (VG) de LVM
- Los volúmenes son **volúmenes lógicos** (LV)
- Sin almacenamiento compartido
- Sin qcow2 ni snapshots

DISK

- Controla un **disco físico** completo
- Los volúmenes son **particiones** del disco

NETFS

- Monta un **directorio de un servidor NFS**
- Los volúmenes son **ficheros de imagen**
- **Almacenamiento compartido** entre hosts

02

Gestión de pools y volúmenes

virsh, qemu-img y herramientas específicas

Dos enfoques para gestionar volúmenes

Sirven para **cualquier tipo de pool**:

CON LIBVIRT (`VIRSH VOL-*` / `VIRT-MANAGER`)

Los mismos comandos en todos los pools:
libvirt crea el volumen **del tipo que corresponde** al pool (fichero de imagen, LV o partición).

CON LA HERRAMIENTA PROPIA DEL POOL

- `dir`, `netfs` : `qemu-img`
- `logical` : `lvcreate`, `lvextend`, `lvremove`
- `disk` : `parted`

Y después `virsh pool-refresh` .

i libvirt no se entera de lo que hacen otras herramientas: tras crear, modificar o borrar un volumen con ellas, hay que ejecutar `virsh pool-refresh <pool>` para que lo detecte.

Gestión de pools de almacenamiento

```
virsh pool-list
virsh pool-info default
virsh pool-dumpxml default

# Crear, construir, arrancar y persistir un nuevo pool
virsh pool-define-as vm-images dir --target /srv/images
virsh pool-build      vm-images
virsh pool-start      vm-images
virsh pool-autostart  vm-images

# Detener y eliminar
virsh pool-destroy  vm-images
virsh pool-delete   vm-images
virsh pool-undefine vm-images
```

Gestión de volúmenes con libvirt

```
virsh vol-list default
virsh vol-list default --details
virsh vol-info prueba1.qcow2 default
virsh vol-dumpxml vol.qcow2 default

# Crear un volumen qcow2 de 10 GB en el pool default
virsh vol-create-as default vol1.qcow2 --format qcow2 10G

# Eliminar un volumen
virsh vol-delete vol1.qcow2 default
```

Gestión de volúmenes con `qemu-img`

En un pool de tipo `dir`, por ejemplo el pool `default`:

```
cd /var/lib/libvirt/images

# Crear imagen qcow2 de 2 GB
qemu-img create -f qcow2 vol2.qcow2 2G

# Ver información de la imagen
qemu-img info vol2.qcow2

# Hacer que libvirt detecte el nuevo fichero
virsh pool-refresh default
```

Creación de MV usando volúmenes existentes

```
virt-install \  
    --virt-type kvm \  
    --name prueba4 \  
    --cdrom ~/iso/debian-11.3.0-amd64-netinst.iso \  
    --os-variant debian10 \  
    --disk vol=default/vol1.qcow2 \  
    --memory 1024 \  
    --vcpus 1
```

Otras formas de indicar el disco:

```
--disk path=/var/lib/libvirt/images/vol1.qcow2  
--disk pool=vm-images,size=10
```

El mismo esquema en un pool `logical`

Con un pool `vg-kvm` que controla el grupo de volúmenes LVM del mismo nombre:

```
# Con libvirt
virsh vol-create-as vg-kvm vol3 5G

# Con LVM
sudo lvcreate -L 5G -n vol3 vg-kvm
virsh pool-refresh vg-kvm
```

 En los dos casos, `virsh vol-list vg-kvm` muestra el volumen y se usa en `virt-install` con `--disk vol=vg-kvm/vol3`.

Cuadro comparativo de pools

POOL	VOLUMEN	GESTIÓN CON VIRSH	HERRAMIENTA PROPIA (+ POOL-REFRESH)	NIVEL	COMPARTIDO	QCOW2 / SNAPSHOTS
dir	Fichero de imagen en un directorio local		qemu-img	Archivo	✗	✓
logical	Volumen lógico (LV) de un grupo de volúmenes LVM	vol-create-as vol-list vol-info	lvcreate , lvextend , lvremove	Bloque	✗	✗
disk	Partición de un disco físico	vol-resize vol-delete	parted	Bloque	✗	✗
netfs	Fichero de imagen en un directorio NFS montado		qemu-img	Archivo	✓	✓

03

Añadir y redimensionar discos

Attach, resize y virt-resize

Añadir nuevos discos a una MV

```
# Añadir disco (funciona también en caliente)
virsh attach-disk prueba4 \
    /srv/images/vol2.qcow2 vdb \
    --driver=qemu --type disk \
    --subdriver qcow2 \
    --persistent

# Eliminar disco
virsh detach-disk prueba4 vdb --persistent
```



`--persistent` guarda el cambio en la configuración XML de la MV para que persista tras reinicios.

Redimensión de discos

Con la MV **parada**:

```
# Con libvirt
virsh vol-resize vol2.qcow2 3G --pool vm-images

# Con qemu-img
sudo qemu-img resize /srv/images/vol2.qcow2 3G
```

Con la MV **en caliente**, y redimensionando después el SF dentro de la MV:

```
virsh blockresize prueba4 /srv/images/vol2.qcow2 3G
resize2fs /dev/vdb      # dentro de la MV
```

Redimensión del SF sin entrar en la MV

- `virt-resize` (paquete `libguestfs-tools`) expande el sistema de ficheros de una imagen **sin arrancar la MV**
- Es la alternativa cuando no se puede, o no interesa, entrar en la MV para ejecutar `resize2fs`
- Siempre trabaja sobre una **copia** de la imagen: nunca modifica directamente el fichero original

⚠ Requiere el paquete `libguestfs-tools` y trabaja siempre sobre una copia del fichero original.

04

Clonación, imágenes cloud y snapshots

Plantillas, cloud-init y instantáneas de MV

Métodos de clonación

A PARTIR DE UNA MV

- `virt-clone` O `virt-manager`
- La MV clonada es **idéntica** a la original
- Problema: misma configuración, hostname, claves SSH...

A PARTIR DE UNA PLANTILLA

Imagen preconfigurada y generalizada (**copia maestra**):

- **Clonación completa (Full)**: copia completa de la plantilla. Requiere el mismo espacio en disco.
- **Clonación enlazada (Linked)**: la plantilla actúa como imagen base de solo lectura (*backing store*). Requiere mucho menos espacio.

Clonación desde una MV con `virt-clone`

```
# Clonación automática (nombre e imagen generados automáticamente)
virt-clone \
    --original prueba4 --auto-clone

# Especificando nombre e imagen destino
virt-clone \
    --original prueba4 \
    --name prueba5 \
    --file /var/lib/libvirt/images/prueba5.qcow2 \
    --auto-clone
```

 La MV clonada es **igual a la original**: mismo hostname, mismas claves SSH, misma configuración. Hay que entrar y modificarla manualmente.

Despliegue automatizado con imágenes cloud

Aunque podemos crear plantillas manualmente con `virt-sysprep` (ver capítulo 5 del curso *Profundización a la virtualización con KVM/libvirt*), en esta asignatura usaremos un enfoque más moderno y estándar:

IMÁGENES CLOUD

Discos base **ya generalizados** y listos para usar como plantilla.

Las distribuyen directamente Ubuntu, Debian, Fedora, CentOS, AlmaLinux...

CLOUD-INIT

Herramienta que **personaliza la MV en su primer arranque**: hostname, usuarios, claves SSH, paquetes, red...

 Una imagen cloud **es una plantilla**: generalizada, sin configuración específica, lista para clonar y personalizar.

Imágenes cloud: plantillas listas para usar

Discos base preconfigurados y optimizados para virtualización: **generalizados** (sin hostname ni usuarios fijos), con **aprovisionamiento ligero**, y **personalizables** en el primer arranque mediante `cloud-init`.

Distribuciones que ofrecen imágenes cloud:

DISTRIBUCIÓN	URL
Ubuntu	https://cloud-images.ubuntu.com
Debian	https://cloud.debian.org/images/cloud/
Fedora	https://alt.fedoraproject.org/cloud/
AlmaLinux	https://repo.almalinux.org/cloud/
Rocky Linux	https://dl.rockylinux.org/pub/rocky/

¿Qué es cloud-init?

`cloud-init` es el estándar para la **inicialización automática** de MV e instancias en la nube (OpenStack, AWS, Azure...) y en entornos KVM/libvirt.

Lee una **configuración en formato YAML** (`cloud-config`) y la aplica en el primer arranque:

CONFIGURACIÓN DEL SISTEMA

- Hostname y FQDN
- Red (IP, DNS, rutas)
- Particiones y montajes

USUARIOS Y SOFTWARE

- Creación de usuarios y grupos
- Inyección de claves SSH
- Instalación de paquetes
- Ejecución de comandos o scripts



Documentación oficial: <https://cloudinit.readthedocs.io/>

Configuración: fichero cloud-config

Creamos un fichero `cloud.yaml` con la configuración deseada:

```
#cloud-config

hostname: ubuntu-vm

package_update: true
package_upgrade: true

chpasswd:
  expire: False
  users:
    - name: root
      password: newpassword
      type: text
    - name: ubuntu
      password: asdasd
      type: text
```

Descargar la imagen cloud

Descargamos la imagen cloud y la guardamos en el pool de libvirt:

```
wget https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img  
sudo mv noble-server-cloudimg-amd64.img /var/lib/libvirt/images
```

Clonación enlazada de la imagen cloud

Creamos una **clonación enlazada** usando la imagen cloud como *backing store*:

```
cd /var/lib/libvirt/images
sudo qemu-img create -f qcow2 \
    -b noble-server-cloudimg-amd64.img \
    -F qcow2 ubuntu2404.qcow2
```

Opcionalmente, ampliamos el tamaño del disco (cloud-init lo redimensionará en el primer arranque):

```
sudo qemu-img resize ubuntu2404.qcow2 20G
```

Crear la MV con cloud-init

```
virt-install --connect qemu:///system \  
    --virt-type kvm \  
    --name ubuntu-vm \  
    --memory 2048 \  
    --vcpus 2 \  
    --os-variant ubuntu24.04 \  
    --disk path=/var/lib/libvirt/images/ubuntu2404.qcow2,format=qcow2,bus=virtio \  
    --import \  
    --cloud-init user-data=/var/lib/libvirt/images/cloud.yaml \  
    --noautoconsole
```

El parámetro `--import` indica que el disco ya tiene SO instalado; `--cloud-init` pasa el fichero de configuración a la MV para que se aplique en el primer arranque.

Acceso tras el primer arranque

Con `--noautoconsole` la MV se crea sin abrir consola gráfica. Una vez arrancada, accedemos por consola serie:

```
virsh console ubuntu-vm
```

 La primera vez que arranca, `cloud-init` aplica la configuración de `cloud.yaml` antes de que puedas iniciar sesión.

Instantáneas de MV (*snapshots*)

- Guardan el estado de disco y memoria en un momento dado, para poder **volver atrás** si algo falla
- Requieren imagen en formato `qcow2`; se pueden crear con la MV apagada o encendida

```
# Crear
virsh snapshot-create-as prueba2 --name "instantánea1" --atomic

# Listar y restaurar
virsh snapshot-list prueba2
virsh snapshot-revert prueba2 instantánea1
```

Otros subcomandos: `snapshot-info`, `snapshot-dumpxml`, `snapshot-delete`.

IV · ALMACENAMIENTO EN QEMU/KVM + LIBVIRT

¡Gracias!

Almacenamiento en QEMU/KVM + libvirt

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 IV